

Traxa Mobile Application

توثيق شامل لمشروع التخرج

تطبيق إدارة الحضور والدرجات الأكاديمية
Flutter + BLoC + WebSocket Realtime

QR + Face ID

REST API

WebSocket

BLoC / Cubit

Dart

+Flutter 3.0

Generated: 2026-05-29 · Version 2.0.0+1

1. نظرة عامة على المشروع

1.1 الفكرة

Traxa هو تطبيق Flutter للهواتف يدير المنظومة الأكاديمية لكلية: **الحضور** (Face Detection + QR)، **الدرجات** (GPA متراكم)، **الجدول**، **إدارة المعيدين (TAs)**، و**تقارير PDF**. التطبيق يتصل بسيرفر Node.js عبر REST + WebSocket للحصول على بيانات لحظية (SESSION_ACTIVATED, GRADE_UPDATED, SEMESTER_CHANGED... إلخ).

1.2 الأدوار (Roles)

الدور	الشاشة الرئيسية	الصلاحيات
-------	-----------------	-----------

عرض الجدول، الدرجات، تعديل البروفایل، QR للحضور	StudentScreen	Student
تفعيل جلسات الحضور، إدارة الدرجات، إدارة المعيدین، التقارير	DoctorScreen	Doctor
حسب صلاحيات الـ Doctor المشرف (per-subject permissions)	DoctorScreen (محدود)	Teaching Assistant (TA)

1.3 الفيتشرز الأساسية

- **Authentication** ثلاثي الأوضاع: Student / Doctor / Admin login مع secure token storage.
- **تفعيل جلسة حضور:** الدكتور يطلب تشغيل الكاميرا عبر Host API → الكاميرا تكشف الوجه → الطلاب يأكدون عبر QR personal code.
- **Real-time updates:** WebSocket reconnect تلقائي مع heartbeat كل 25 ثانية.
- **GPA Calculator:** حساب GPA per-semester / per-level / cumulative مع credit hours.
- **PDF Reports:** توليد تقارير حضور كاملة بصور الحالة (Present/Pending/Absent).
- **Offline Cache:** أول مرة → API، بعدها cache مدته ساعة عبر SharedPreferences.
- **Server URL Override:** المستخدم يقدر يغير IP السيرفر من شاشة Login (مفيد للتطوير المحلي).
- **Theme toggle:** Dark / Light / System persisted.
- **Skeleton loading + Shimmer effect** أثناء الـ Loading.
- **Face ID + QR Code attendance flow** مع timing (مدة الـ confirmation).

2. هيكلية المشروع (Project Structure)

2.1 الشجرة العامة

```
lib/
├─ main.dart                # AuthGate routing + نقطة البداية
├─ core/                    # Foundation layer
│   ├── api_service.dart    # Legacy HTTP facade (يستدعي ApiEndpoints)
│   ├── constants.dart      # Backwards-compatible facade
│   ├── helpers.dart        # GPA + utility functions
│   ├── logger.dart         # AppLogger (debug/info/warn/error)
│   ├── theme.dart          # AppTheme.lightTheme / darkTheme
│   ├── pdf_report_service.dart # PDF generation for attendance reports
│   ├── constants/          # api_endpoints, storage_keys, app_strings
│   ├── di/                 # GetIt service locator + AppProviders
│   ├── env/                # AppEnv, AppFlavor, ServerConfig
│   ├── exceptions/         # Typed exceptions (Network/Server/Auth...)
│   ├── extensions/         # context_x, datetime_x, string_x
│   ├── result/             # sealed Result<T> (Success/Failure)
│   ├── style/              # app_colors, app_dimensions, app_text_styles
│   └─ utils/               # error_handler
├─ cubit/                   # State Management (BLoC)
│   ├── auth/ (auth_cubit + auth_state)
│   ├── data/ (data_cubit + data_state)
│   ├── theme/ (theme_cubit)
│   └─ shared/ (loading_state + error_state)
├─ models/                  # Plain Dart classes + fromJson/toJson
│   ├── user.dart, student.dart, doctor.dart
│   ├── teaching_assistant.dart, subject.dart, lecture.dart, section.dart
│   ├── grade.dart, attendance.dart, qr_code.dart
├─ repositories/            # API abstraction layer
│   ├── base_repository.dart # guard() + fromLegacyResponse()
│   ├── auth_repository.dart
│   ├── student_repository.dart, doctor_repository.dart
│   ├── grades_repository.dart, attendance_repository.dart
│   ├── ta_repository.dart, system_repository.dart
│   └─ repositories.dart     # Barrel export
├─ services/
│   ├── secure_storage_service.dart # FlutterSecureStorage wrapper
│   ├── token_holder.dart          # ValueListenable<String?> token
│   ├── cache_service.dart         # 1-hour offline cache via SharedPrefs
│   └─ websocket_service.dart      # Singleton WS + auto-reconnect
└─ features/                  # UI Screens (split by feature)
```

```

|   ├── auth/ (login_screen, server_url_dialog)
|   ├── doctor/ (doctor_screen + 6 sections)
|   └── student/ (student_screen + 4 sections)
└── widgets/ # Reusable widgets
    ├── academic_year_chip, app_skeleton, app_snack
    ├── custom_card, error_widget, gradient_button
    ├── lecture_card, loading_indicator, profile_avatar
    ├── stat_card, theme_toggle_button, toast_message
    └── ws_connection_dot

```

2.2 المعمارية (Architecture)

الـ Architecture يتبع طبقات واضحة (Layered):

الطبقة	المهمة	أمثلة
UI (features/widgets)	عرض البيانات + تلقي input	LoginScreen, DoctorOverview
State (cubit/)	إدارة الحالة + ربط Repository بالـ UI	AuthCubit, DataCubit
Repository (repositories/)	تجميع API calls + تحويلها لـ <Result<T	()AuthRepository.login
Service (services/)	خدمات منخفضة المستوى (WS, storage, cache)	WebSocketService, SecureStorage
Core (core/)	أدوات مساعدة + ثوابت + theme	AppEnv, AppLogger, AppTheme
Model (models/)	تمثيل البيانات (immutable)	User, Grade, Subject

لماذا Layered Architecture؟

البديل (Putting everything in StatefulWidget) كان يعمل لكن بيخلي الكود غير قابل للاختبار ومتشاك. الطبقات بتخلي كل جزء قابل للاستبدال (مثلاً لو غيرنا BLoC بـ Riverpod، الـ Repository هيبقى زي ما هو).

3. الحزم (Packages) ولماذا اختيرت

الحزمة	الإصدار	الاستخدام	البدايل	ليه اختيرنا
flutter_bloc	8.1.3 [^]	Cubit State management عبر (نسخة مبسطة من BLoC)	provider, riverpod, getx, mobx	Cubit أبسط الكامل، وأ riverpod من بديل قوي community consumer e-effects
get_it	7.7.0 [^]	Service Locator (Dependency Injection)	provider, riverpod, injectable	يفصل إنشاء repositories شجرة الـ locator يسجل كل singleton بدء التطبيق
equatable	2.0.5 [^]	مقارنة الـ Cubit States (يمنع إعادة بناء widget لو الحالة ما تغيرت)	كتابة == و hashCode يدويًا، freezed	أبسط بكثير يدوي، وأخذ (اللي يحتا generation
http	1.1.0 [^]	REST API للـ HTTP requests	dio, chopper, retrofit	الرسمي ه خفيف، يك لو احتجنا معقدة أو .tokens
http_parser	4.0.2 [^]	multipart uploads لـ MediaType (رفع avatar مثلاً)	—	مكتبة مك ملفات ص
web_socket_channel	2.4.0 [^]	اتصال WebSocket للتحديثات اللحظية	socket_io_client, pusher	السيرفر ي WebSocket socket.IO package socket.IO تغيير بروت

key-value Hive/Isar تخزين لكن دلوقة	hive, isar, sqflite	تخزين البيانات البسيطة (cache,) (theme, server URL	2.2.2 [^]	shared_preferences
يستخدم Keychain مشفر ضروري للـ	shared_preferences (tokens للـ آمن)	تخزين الـ JWT token + user data مشفر	9.0.0 [^]	flutter_secure_storage
API من inconfusion للاستخدام	printing, syncfusion_flutter_pdf	توليد تقارير PDF للحضور	3.10.8 [^]	pdf
الـ Standard للملفات	—	الحصول على مجلدات النظام لحفظ الـ PDF	2.1.2 [^]	path_provider
يفتح الملف الافتراضي	url_launcher, share_plus	فتح ملف PDF بعد توليده	3.5.10 [^]	open_file
الأبسط لـ file_picker	file_picker, photo_manager	اختيار صور للـ avatar من الجاليري/ الكاميرا	1.0.7 [^]	image_picker
يقلل الـ width تجربة الـ تحميل.	Image.network (بدون cache)	عرض صور البروفايل من السيرفر مع cache + placeholder	3.3.1 [^]	cached_network_image
الأكثر استـ	qr.flutter barcode_widget (مهمل),	توليد QR code للطالب (شاشة Student Profile)	4.1.0 [^]	qr_flutter
أقوى من S bridge)	webview_flutter	(مدرج للاستعداد لشاشات WebView مستقبلية)	6.0.0 [^]	flutter_inappwebview
الـ Standard Flutter.	—	طلب أذونات الكاميرا والميكروفون والتخزين	11.3.0 [^]	permission_handler
يعطي تنـ الـ أيقونات أكاديمية).	Material Icons فقط	أيقونات إضافية (FontAwesomeIcons.userGraduate) (إلخ)	10.7.0 [^]	font_awesome_flutter
أبسط API abled: loading)	shimmer, flutter_skeleton	Shimmer loading لشاشات الـ List	2.1.3 [^]	skeletonizer
API نظيف الفرق بين صغيرة وـ	MediaQuery يدويًا, sizer	Responsive sizing عبر .h / .w .r / .sp	5.9.3 [^]	flutter_screenutil

الـ standard	—	لمقارنة DeepCollectionEquality في الـ Maps Models	1.19.1^	collection
افتراضي د	—	أيقونات iOS-style	1.0.6^	cupertino_icons

الـ Dev Dependencies

- **flutter_lints** ^3.0.0 — قواعد Lint رسمية لجودة الكود.
- **flutter_test** — اختبارات الـ widgets والـ unit tests.

4. ملف pubspec.yaml

```
name: traxa_mobile
description: A new Flutter project.
publish_to: 'none'
version: 2.0.0+1

environment:
  sdk: '>=3.0.0 <4.0.0'

dependencies:
  flutter: { sdk: flutter }
  cupertino_icons: ^1.0.6
  http: ^1.1.0
  http_parser: ^4.0.2
  shared_preferences: ^2.2.2
  flutter_secure_storage: ^9.0.0
  font_awesome_flutter: ^10.7.0
  cached_network_image: ^3.3.1
  image_picker: ^1.0.7
  flutter_bloc: ^8.1.3
  pdf: ^3.10.8
  path_provider: ^2.1.2
  open_file: ^3.5.10
  flutter_inappwebview: ^6.0.0
  permission_handler: ^11.3.0
  equatable: ^2.0.5
  qr_flutter: ^4.1.0
  web_socket_channel: ^2.4.0
  collection: ^1.19.1
  skeletonizer: ^2.1.3
  flutter_screenutil: ^5.9.3
  get_it: ^7.7.0

dev_dependencies:
  flutter_test: { sdk: flutter }
  flutter_lints: ^3.0.0

flutter:
  uses-material-design: true
  assets:
    - icons/
```


الشرح

- **name:** اسم ال package (يستخدم في ال imports عبر `.../package:traxa_mobile`)
- **version:** 2.0.0 هو ال user-facing version، +1 هو ال build number (للـ APK / Play Store).
- **environment.sdk:** يفرض 3 Dart + (لاستخدام sealed classes في `<Result<T` و pattern matching).
- **assets:** فولدر /icons يحتوي على لوجو التطبيق والأيقونات المخصصة.

5. ملف AndroidManifest.xml

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
  <uses-permission android:name="android.permission.CAMERA" />
  <uses-permission android:name="android.permission.RECORD_AUDIO" />
  <uses-permission android:name="android.permission.INTERNET" />

  <application
    android:label="Traxa"
    android:name="${applicationName}"
    android:icon="@mipmap/ic_launcher"
    android:usesCleartextTraffic="true">
    <activity
      android:name=".MainActivity"
      android:exported="true"
      android:launchMode="singleTop"
      android:taskAffinity=""
      android:theme="@style/LaunchTheme"
      android:configChanges="orientation|keyboardHidden|keyboard|screenSize|smallestScreenSize"
      android:hardwareAccelerated="true"
      android:windowSoftInputMode="adjustResize">
      <meta-data android:name="io.flutter.embedding.android.NormalTheme" android:resource="@style/NormalTheme" />
      <intent-filter>
        <action android:name="android.intent.action.MAIN" />
        <category android:name="android.intent.category.LAUNCHER" />
      </intent-filter>
    </activity>
    <meta-data android:name="flutterEmbedding" android:value="2" />
  </application>
</manifest>
```

التعديلات والشرح

السطر	الشرح	السبب
CAMERA	صلاحية الكاميرا	image_picker يستخدمها لرفع صور البروفايل. (AI Face Detection يحصل على السيرفر مش الموبايل).
RECORD_AUDIO	صلاحية الميكروفون	flutter_inappwebview يطلبها افتراضياً (حتى لو ما استخدمناها)

فعلياً).		
ضروري لأي تطبيق يتصل بـ API/WebSocket.	صلاحية الإنترنت	INTERNET
للسيرفرات المحلية أثناء التطوير (مثلاً http://192.168.1.17). في الـ Production السيرفر يستخدم HTTPS.	السماح بـ HTTP بدون SSL	"usesCleartextTraffic"=true
يحافظ على الـ state بدون restart عند استقبال notification.	منع فتح Activity جديدة لو موجودة	"launchMode"=singleTop
يجعل Flutter يتعامل معها بدل ما Android تعمل restart.	منع إعادة إنشاء الـ Activity عند تغير الـ orientation/locale	configChanges
يمنع التغطية على حقول الإدخال (مهم في شاشة Login).	تقليل حجم الـ Layout عند فتح الكيبورد	"windowSoftInputMode"=adjustResize

6. main.dart — نقطة البداية

الكود

```
Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await ServerConfig.load(); // السيرفر المحفوظ URL تحميل (1)
  HttpOverrides.global = MyHttpOverrides(); // (2) للسيرفر المحدد SSL ثقة بـ
  await initServiceLocator(); // (3) GetIt + Cubits تهيئة كل
  runApp(const MyApp());
}
```

التفسير

1. **ServerConfig.load()**: يقرأ من SharedPreferences الـ URL اللي حفظه المستخدم آخر مرة (لو غير الـ default).
2. **MyHttpOverrides**: يجعل التطبيق يثق بشهادات SSL للسيرفر معين فقط (للسماح بـ self-signed certs أثناء التطوير).
3. **initServiceLocator**: يسجل كل الـ Repositories والـ Cubits في GetIt، ثم يستدعي ()AuthCubit.init لاستعادة الجلسة، ثم ()WebSocketService.connect.

AuthGate_

هو widget داخلي يستمع لـ AuthCubit و:

- لو المستخدم authenticated وهو **doctor** أو **DoctorScreen** → **teaching-assistant**
- لو **student** → **StudentScreen**
- لو غير مسجل → **LoginScreen**

عند تغير الـ user، يستدعي **DataCubit.fullReload()** تلقائياً لتحديث البيانات بناءً على الـ user الجديد.

ScreenUtilInit

بناءً على شاشة الجهاز. designSize: **Size(393, 873)** — مقاسات تصميم Pixel-style. كل **w / .h / .sp.** في الكود يحسب نسبياً

TextScaler clamping

نطبق **textScaler.clamp(1.0, 1.2)** لمنع المستخدمين اللي رافعين الـ system font size من تكبير الـ UI (يمنع overflow).

7. طبقة Core

core/api_service.dart 7.1

Legacy HTTP facade يحتوي على كل الـ HTTP calls مباشرة (login, getStudents, getGrades,...recordQRCodeScan). كل دالة ترجع `<Map<String, dynamic>` أو `<List<dynamic>` خام.

الدوال الرئيسية

المجموعة	الدوال
Token	initToken, setToken, clearToken, getToken, hasValidToken
Auth	login (admin), studentLogin, doctorLogin, logout, changeStudentPassword
Public read	getStudentsPublic, getDoctorsPublic, getSubjectsPublic, getLecturesPublic
Authenticated read	getStudents, getDoctors, getSubjects, getLectures, getLecturesBySemester
Sections	getSections, getSectionsBySemester
TAs	getTeachingAssistants, getTeachingAssistantsForDoctor, getTAPermissions, updateTAPermissions, getTASubjectPermissions, updateTASubjectPermissions
System	getCurrentSemester, getCurrentAcademicYear, persistAcademicYear
Grades	getStudentGrades, getStudentGradesWithToken, checkGradesStatus, getGradeDistribution
QR	getStudentQRCode, recordQRCodeScan, getDoctorQRCodes
Enrollment	getStudentEnrollmentStatus, getAvailableSubjectsForEnrollment, registerForSubject, dropSubject
Attendance	getAttendance
Avatar	forceRemoveAvatar (يرفع PNG فارغ ثم يحذف)
Subject TA	assignTAToSubject, removeTAFromSubject

ملاحظة مهمة: هذا الملف هو طبقة قديمة. الكود الجديد يستخدم Repositories اللي بتلف حوالها وترجع `<Result<T>` بدل من Map خام. تدريجياً، كل callsite بيتنقل لـ Repository.

core/env/server_config.dart 7.8

يدير URL السيرفر:

- **compileTimeDefault:** `https://traxa-system.online` (production server JI).
- **load():** يقرأ override المحفوظ.
- **setUrl():** يحفظ override جديد.
- **normalize_():** يضيف `https://` لو غير موجود + يحذف `/` الأخيرة.
- **validate():** يتأكد إن URL parseable و host غير فارغ.

core/exceptions/app_exception.dart 7.9

سلسلة typed exceptions:

- **AppException** (abstract) — الأب.
 - **NetworkException / ServerException / TimeoutException** — أخطاء الشبكة.
 - **UnauthorizedException / SessionExpiredException / InvalidCredentialsException** — أخطاء .auth
 - **AccountLockedException** — مع `remainingAttempts` و `lockedAt`.
 - **ValidationException** — مع `fieldErrors: Map`.
 - **UnknownAppException** — fallback
- الفايدة: الـ `if (e is SessionExpiredException) → navigateToLogin` تعمل بدل ما تطابق نصوص.

/core/extensions 7.10

المحتوى	الملف
BuildContext: <code>theme, textTheme, colors, isDark, adaptiveCard, ()screenWidth, showSnack(), dismissKeyboard</code> اختصارات	context_x.dart
<code>isToday, isYesterday, iso8601Date, hhmm, timeAgo</code>	datetime_x.dart
<code>capitalize, titleCase, initial, nullIfBlank, isBlank/isNotBlank</code>	string_x.dart
Barrel export للملفات الثلاثة.	extensions.dart

core/result/result.dart 7.11

<**Sealed class Result**> يستبدل النمط القديم `{... : 'success': bool, 'error'}`.

```
sealed class Result<T> {
    bool get isSuccess;
    T? get valueOrNull;
    AppException? get exceptionOrNull;

    R when<R>({
        required R Function(T) success,
        required R Function(AppException) failure,
    });

    Result<R> map<R>(...);
    Future<Result<R>> flatMap<R>(...);
}

class Success<T> extends Result<T> { final T value; }
class Failure<T> extends Result<T> { final AppException exception; }
```

المميزات: ما حدثش يقدر ينسى يعالج حالة ال failure (compile-time check بفضل sealed).

/core/style 7.12

المحتوى	الملف
secondary=#0EA5E9, success=#10B981, primary=#8B5CF6 (بنفسجي), warning=#F59E0B, error=#EF4444. ألوان ال Dark/Light backgrounds و .texts	app_colors.dart
spacing scale (s4..s32, v4..v24), radius scale (sm..xl), card/button paddings كلها تستخدم <code>w / .h / .r</code> من <code>screenutil</code> .	app_dimensions.dart
headlineLarge/Medium, titleLarge/Medium/Small, bodyLarge/Medium/Small, appBar — كلها تستخدم <code>.sp</code> .	app_text_styles.dart

core/theme.dart 7.13

AppColors + ThemeData من ال بينوا `AppTheme.darkTheme` و `AppTheme.lightTheme` في `AppTextStyles` `buildTheme_()` داخلي يحدد:

- scaffoldBackgroundColor, cardColor
- cardTheme (radius lg + border)
- appBarTheme (transparent, custom title style)
- inputDecorationTheme (filled + outline borders)
- elevatedButtonTheme (primary color + radius md)
- textTheme (مربوط بـ AppTextStyles)

bottomNavigationBarTheme •

adaptive colors لل السريع للوصول `ThemeContextExtension` — extensions
`(context.adaptiveCardColor)`

core/logger.dart 7.14

`AppLogger` بأربع مستويات (debug, info, warn, error). في ال prod release يطبع errors فقط ويستدعي `onError` callback لإرسالها ل Crashlytics/Sentry لو احتجنا. `logDebug()` دالة مَعْلَمَة legacy `Deprecated`.

core/helpers.dart 7.15

دوال أكاديمية:

- lookups — `findStudentSafely` / `findDoctorSafely` / `findSubjectSafely` آمنة بدون رمي exceptions.
- `()getTodayDayName` — اسم اليوم بالإنجليزي.
- `calculateGPA(grades, subjects)` — مجموع (gradePoints × credits) ÷ مجموع credits.
- `calculateSemesterGPA` / `calculateLevelGPA` / `calculateCumulativeGPA`
- `calculateEarnedCredits` — الكريديتس المنجحة فقط (total ≥ 50).
- `calculateRegisteredCredits` — كل الكريديتس المسجلة.
- `calculateTotalGradePoints`
- `countPassedSubjects` / `countFailedSubjects`
- `getGradeLabel(gpa)` — Excellent/Very Good/Good/Satisfactory/Pass/Needs Improvement

core/utils/error_handler.dart 7.16

`ErrorHandler.getMessage(e)` يحول error خام إلى نص "No user-friendly (SocketException → "Invalid credentials"...)
 401 → "internet connection", كمان فيه `showErrorSnackBar` و `showSuccessSnackBar`.

core/pdf_report_service.dart 7.17

: `PdfReportService.generateAndOpen(context, report)`

1. يستخرج بيانات التقرير (subject, doctor, start/end, students list, level, department).
2. يحسب `presentCount`, `pendingCount`, `absentCount`, `attendanceRate`.
3. يبني صفحة A4 بـ:
 - Header (logo Traxa + اسم ال subject + التاريخ).
 - Summary card (Start/End/Level/Dept + إحصائيات الحضور).

- Students table (#, ID, Name, Status, Face ID, QR, Duration) ○
- Footer (page X of Y) ○
- 4. يحفظ الملف في `getApplicationDocumentsDirectory` + ينسخه في `Downloads` (Android).
- 5. يفتحه بـ `OpenFile.open`.

دوال داخلية: `header, _summary, _item, _studentsTable, _time12_` (يتعامل مع ISO datetime أو `dateOnly_`، "string "HH:mm مع/بدون (AM/PM).

منطق Face/QR: العمود "Face ID" نعم/لا حسب وجود QR. `face_detected_at` " نعم/لا حسب `qr_scanned_at` أو `Duration`. `confirmedByQR == true` = الفرق بين الوقتين.

core/di/service_locator.dart 7.18

يستخدم (singleton container) **GetIt**. دالة `initServiceLocator()`:

1. تستدعي `ApiService.initToken()` (legacy in-memory token).
2. تسجل `TokenHolder` وتحمل token المحفوظ.
3. تسجل `WebSocketService.instance`.
4. تسجل كل الـ Repositories الـ 7 (Auth, Student, Doctor, Grades, Attendance, Ta, System).
5. تسجل الـ Cubits الثلاثة (Auth, Data, Theme).
6. تستدعي `AuthCubit.init()` لاستعادة الجلسة قبل إرسال أي API.
7. تستدعي `DataCubit.init()` لتحميل البيانات.
8. تستدعي `WebSocketService.connect()`.

كل registration بـ `guard` `isRegistered` لمنع الـ duplicate في hot restart.

core/di/app_providers.dart 7.19

يلف الـ MaterialApp في `MultiBlocProvider` لكن باستخدام `value constructor`. لأن الـ Cubits بقت ملك GetIt مش الـ widget tree. ده يضمن إن `<context.readX()` يشتغل في أي شاشة، وفي نفس الوقت ما نعملش re-create الـ Cubits عند الـ hot restart.

8. طبقة State Management (Cubit)

8.1 Cubit ليس BLoC الكامل؟

Cubit أبسط: بدل ما تعمل Event classes لكل action، أنت بتنادي method مباشرة. مناسب للمشروع لأن الـ events بسيطة (login, refresh, updateGrade). BLoC الكامل أفضل لو احتجنا event transformation معقدة (debounce, throttle).

8.2 cubit/shared/loading_state.dart

فيه `enum LoadingStatus { initial, loading, loaded, error }` + class wrapper `LoadingState` .getters: `isLoading, isLoading, hasError`

8.3 cubit/shared/error_state.dart

Class بسيط بـ `hasError, message` . (الكود الحديث يستخدم `LoadingState.error` بدلاً منه).

8.4 cubit/auth/auth_cubit.dart + auth_state.dart

AuthState (Equatable): `user, token, isAuthenticated, loadingState` + factories: `initial / loading / success / error / loggedOut`

:AuthCubit methods

- **init() / checkAuth** — يحاول استعادة الجلسة من SecureStorage. لو نجح: `emit success + connect` .WebSocket
- **login(username, password, isStudent)** — يوحد الـ `entry` لـ `studentLogin / doctorLogin` حسب الـ `toggle`
- **studentLogin(studentId, password, isFirstLogin)** — مع `flag` للـ `first-login` (الطالب لازم يغير `(password)`
- **doctorLogin(username, password)**
- **updateUserPermissions(newPerms)** — يحدّث صلاحيات الـ `TA` في الـ `state` + يحفظ في `SecureStorage`
- **logout()** — يمسح الـ `token` + يقطع الـ `WebSocket` + `emit loggedOut`
- **refreshUserData()** — يجيب بيانات الـ `user` الحديثة من السيرفر (TA يتم تجاوزه عشان بيانات الـ `TA` بتيجي من اللوجين).

cubit/data/data_cubit.dart + data_state.dart 8.5

DataState: قوائم + students/doctors/subjects/lectures/sections/grades/attendance/TAs
.currentSemester + currentAcademicYear + loadingState

Helper methods داخل: getSubjectsForDoctor, getLecturesForDoctor, getSubjectsForStudent
getSectionsForStudent, getGradesForStudentAndSemester, (department و level بالفلتر)
.getAllGradesForStudent, semesterDisplay

:DataCubit methods

- **init() / loadSystemData** — يحمل semester + academic year الحالي.
- **loadAllData() / fullReload** — يجيب كل البيانات (students, doctors, subjects, lectures, sections, TAS) بالتوازي عبر `Future.wait`، يفلتر حسب السمستر الحالي، يحفظ في `cache`.
- **emitFromCache()** — fallback_ لو `network failed`.
- **loadStudentGrades / loadStudentGradesWithToken**
- **getGradesForSemester(semester)**
- **checkGradesStatus(studentId, token)** — debug endpoint
- **loadAttendance(token)**
- **fetchTAsForDoctor(doctorId)**
- **fetchTAPermissions / updateTAPermissions**
- **refreshForNewSemester** — يعيد التحميل عند تغير السمستر.
- **updateGrade(updatedGrade)** — تحديث grade واحد (يفيد لحظياً عند WebSocket `GRADE_UPDATED`).
- **clearData()** — Reset to initial

cubit/theme/theme_cubit.dart 8.6

ThemeState فيه `_loadTheme` (system/light/dark). `ThemeMode` () يقرأ من `SharedPreferences` key
`setTheme(mode)` . `theme_mode` يحفظ + `toggleTheme` . `emit` () يقلب بين `dark` و `light`.

9. طبقة Models

كل ال (Models immutable (`final` fields + `const` constructors لها `fromJson` / `toJson` / `copyWith` + `operator==` / `hashCode` + `toString`.

User 9.1

يغطي كل الأدوار (student/doctor/TA/admin) عبر `role` + `userType`. فيه: `id`, `username`, `name`, `email`, `supervisorDoctorId` (لـ TA), `department`, `level`, `permissions`, `taPermissions` (per-subject), `tald`.

Getters: `isDoctor`, `isStudent`, `isTeachingAssistant`, `effectiveDoctorId` (الـ TA يستخدم doctor المشرف لما يجب subjects).

Helpers: `hasTAPermission(key)`, `canActivateSessionForSubject(subjectId)`.

Student 9.2

`id`, `name`, `studentId` (الكود الجامعي), `faceData`, `department`, `level` (Map لبيانات الوجه), `academicYear`.

Doctor 9.3

`id`, `name`, `username`, `email` (model بسيط).

TeachingAssistant 9.4

`id`, `name`, `username`, `email`, `supervisorDoctorId`, `permissions` (Map), `assignedSubjectIds` (List).

Helpers: `hasPermissions`, `hasPermission(key)`, `isAssignedToSubject`.

Subject 9.5

`id`, `code`, `name`, `doctorId`, `doctorName`, `level`, `semester`, `department`, `credits/creditHours`, `tald`, `taName`.

fallback — `totalCreditHours` ذكي: 3 → `creditHours` → `credits` (الـ default الجامعي).

Lecture 9.6

id, subjectId, subjectName, doctorId, doctorName, level, department, day, timeslotId, timeDisplay, locationId, locationName, active

Section 9.7

يمثل sections (تمارين معيدين). فيه parsing ذكي لأن السيرفر بيرجع keys مختلفة (`ta_name` أو `taName` أو `teaching_assistant_name` ...). الكود يجرب كل الأسماء المحتملة.

getter `timeDisplay` : لو موجود يستخدمه، وإلا يبني `$startTime - $endTime`.

static helper `Section.isSection(json)` — يحدد لو ال entry هي section ولا lecture (عبر type أو `is_section` flag).

Grade 9.8

id, studentId, studentName, subjectId, subjectName, doctorId, level, department, midterm, oral, practical, attendance, assignment, finalExam, total, semester, academicYear, isVisible

:Getters

- **gradeLetter** — A+/A/A-/B+/.../F (total بناءً على 12-point scale).
- **gradePoints** — 4.0 / 3.7 / 3.4 ... 0.0 (للحساب GPA).
- **isPassed** — total ≥ 50
- **gradeColor** — لون لعرض ال percent (أخضر/أصفر/برتقالي/أحمر).

AttendanceRecord 9.9

id, sessionId, lectureId, studentId, studentName, studentIdNumber, subjectName, status (confirmed/pending/absent), confirmedBy, confirmedAt, date, time

.Getters: `isPresent`, `isPending`

QrCode 9.10

id, studentId, studentName, studentCode, studentLevel, studentDepartment, qrData (raw), encodedData, generatedAt, createdAt, updatedAt, scanCount, lastScannedAt, version

Getters: `isExpired` (أكثر من 30 يوم), `shortCode` (أول 8 chars).

10. طبقة Repositories

BaseRepository 10.1

`guard<T>(action)` — يلف أي async call ويحوّل ال exceptions ل `<Result<T>`:

- `AppException` → `Failure` نفسها.
- `SocketException` → `NetworkException`
- `http.ClientException` → `NetworkException`
- `async.TimeoutException` → `TimeoutException`
- أي حاجة ثانية → `UnknownAppException`

`fromLegacyResponse(response)` — يحوّل ال legacy Map response ل `Result`، مع mapping للأخطاء المعروفة (locked / invalid credentials / session expired).

AuthRepository 10.2

- `restoreSession()` — يقرأ `token + userData` من `SecureStorage`. ترجع null لو غير موجود (مش error).
- `login(username, password)` — admin login. يضيف `role=admin` افتراضياً.
- `studentLogin / doctorLogin`
- `refreshUser(current, token)` — يجيب `user` حديث من السيرفر (TA يتم تجاوزه).
- `logout()` — يمسح `token` من `ApiService` + `SharedPreferences` + `SecureStorage`.
- `persistSession(user, token)` — helper_ لحفظ الجلسة.

StudentRepository 10.3

- `getAllStudents, getPublicStudents`
- `changePassword(studentId, newPassword, token)`
- `getQrCode, recordQrScan`
- `getEnrollmentStatus, getAvailableSubjects`
- `registerForSubject, dropSubject`

DoctorRepository 10.4

- `getAllDoctors, getPublicDoctors`
- `getSubjects, getPublicSubjects`
- `getLectures, getLecturesBySemester`

- `.getQrCodes(doctorId, token)`
- `.assignTA, removeTA (per subject)`
- `.deleteAvatar(kind, id, token)`

GradesRepository 10.5

- `.getStudentGrades, getStudentGradesWithToken`
- `.checkStatus (debug)`
- `.getDistribution(doctorId, subjectId, token)` — يرجع تقسيم الدرجات
`.default 10/5/20/5/10/50 = 100` `.(midterm/oral/practical/attendance/assignment/final)`

AttendanceRepository 10.6

دي أكبر repository، فيها كل ال session lifecycle:

- `getAll(token)` — كل سجلات الحضور (legacy).
- `requestCameraHost(token, sessionId, lectureId, doctorName)` — يطلب السيرفر بيعت WebSocket message لل host-listener.
- `waitForCameraStarted(sessionId, token, timeout=90s)` — polling ل `api/attendance-/` `sessions/:id`. تنتظر 90 ثانية لأن face-api.js على ال host بيأخذ وقت لتحميل ال models من GitHub CDN.
- `beginQrPhase(sessionId, token, durationMinutes=30)` — يبدأ مرحلة ال QR (بعد ال face detection).
- `deleteSession(sessionId, token)` — يرجع statusCode للتمييز بين 200/404.
- `heartbeat(sessionId, token)` — fire-and-forget كل ثواني للحفاظ على ال session.
- `getSessionData / pushSessionData` — لحالة ال sync.
- `listReportsByDoctor / saveReport / getReportBySession / deleteReport`
- `listSubjects / getEnrolledStudents`

TaRepository 10.7

- `getAll` (كل المعيدين)، `getForDoctor` (معيدي دكتور).
- `getPermissions / updatePermissions` (عام لل TA).
- `getSubjectPermissions / updateSubjectPermissions` (per-subject، ال flags هي `ta.session.activate` و `ta.grades.manage`).

SystemRepository 10.8

- `getCurrentSemester` (1 أو 2).

- `getCurrentAcademicYear` (مثلاً "2027-2026").
- `.getAcademicSettings(token)`
- `.getSections / getSectionsBySemester`

11. طبقة Services

SecureStorageService 11.1

يستخدم `flutter_secure_storage` مع `Android EncryptedSharedPreferences` و `iOS Keychain` (first_unlock accessibility). فيه `_write/_read/_delete` helper internal مع error handling (لو القراءة فشلت، يحذف القيمة المعطوية).

المفاتيح: `auth_token_secure, user_data_secure, refresh_token_secure`.

API: `saveToken/getToken/hasToken/deleteToken, saveUserData/getUserData/deleteUserData, saveRefreshToken/getRefreshToken/deleteRefreshToken, clearAll`

TokenHolder 11.2

يلف ال token في `<?ValueNotifier<String` ليبقى `observable`. الفائدة: أي widget يقدر يستمع للتغير بدون polling. فيه `authHeader` getter يرجع `<Map<String, String` جاهزة للاستخدام في HTTP requests.

CacheService 11.3

يحفظ `students/doctors/subjects/lectures` في `SharedPrefs` مع `timestamp`. `loadAllData(ignoreExpiry: false)` ترجع null لو ال cache أقدم من ساعة. لما ال network يفشل، ال DataCubit يستدعيها بـ `ignoreExpiry: true` لتقديم أحدث snapshot للمستخدم.

WebSocketService (Singleton) 11.4

أهم خدمة في التطبيق للـ `realtime`. ال instance واحد عام (`WebSocketService.instance`) لأن مفيش معنى لفتح أكثر من اتصال.

ال Streams (broadcast)

ال Stream	ال Type	متى يتم البث
<code>dataChangeStream</code>	<code>Map</code>	أي <code>DATA_CHANGE</code>
<code>semesterStream</code>	<code>int</code>	<code>SEMESTER_CHANGED</code>
<code>academicYearStream</code>	<code>String</code>	<code>ACADEMIC_YEAR_CHANGED</code>

DATA_CHANGE entity=grade أو GRADE_UPDATED	Map	gradeUpdateStream
attendance-session/created أو SESSION_ACTIVATED	Map	sessionActivatedStream
ended action أو SESSION_ENDED	Map	sessionEndedStream
REPORT_SAVED	Map	reportSavedStream
REGISTRATION_APPROVED	List	registrationApprovedStream
LEVELS_PROMOTED	Map	levelsPromotedStream
teaching-assistant/permissions-updated	Map	taPermissionsStream

ال Connection lifecycle

1. **connect()**: يقرأ token + userData، ينشئ HttpClient بـ badCertificateCallback للسيرفر المعروف، يتصل بـ WebSocket مع 12s timeout.
2. يبعث **REGISTER** فوراً مع userRole (mobile-doctor / mobile-student), clientId, userType.
3. يبدأ **heartbeat** كل 25 ثانية.
4. لو الاتصال انقطع: **exponential backoff** يبدأ من 5 ثواني للوصول لأقصى 30 ثانية، حتى 10 محاولات.
5. switch_ **handleMessage**: على **type** للرسائل القياسية + **handleDataChange_** للرسائل DATA_CHANGE التفصيلية (entity + action).

أنواع الرسائل المدعومة

HEARTBEAT_ACK, DATA_CHANGE, (يرد بـ PONG), CONNECTION_ESTABLISHED, REGISTERED, PING, SEMESTER_CHANGED, ACADEMIC_YEAR_CHANGED, SESSION_ACTIVATED, SESSION_ENDED, REPORT_SAVED, GRADE_UPDATED, REGISTRATION_APPROVED, LEVELS_PROMOTED, FULL_SYNC, TOKEN_EXPIRED.

12. طبقة (Screens) Features

12.1 features/auth/login_screen.dart

الشاشة الأولى. فيها:

- **Role toggle** (Student / Doctor) عبر ValueNotifier.
- **Form** مع username + password (مع obscure toggle).
- **"Sign In"** GradientButton مع loading state.
- **Server URL chip** (يفتح showServerUrlDialog لتغيير URL السيرفر).
- **BlocListener<AuthCubit** < BlocListener لعرض snackbar للخطأ.

بعد نجاح الـ AuthCubit، login يرسل state Authenticated، الـ AuthGate (في main.dart) يحول تلقائياً للشاشة المناسبة.

12.2 features/auth/server_url_dialog.dart

Dialog لإدخال URL السيرفر يدوياً (مثلاً `https://192.168.1.17:3443` للتطوير المحلي أو `https://traxa-system.online` للـ production). يستخدم `ServerConfig.setUrl()` ويرجع true لو تم الحفظ.

12.3 features/doctor/doctor_screen.dart

الـ Shell الرئيسي للدكتور/المعيد. فيه:

- **IndexedStack** بـ 5 sections (Home, Subjects, Attendance, Reports, Profile) لحفاظ على state كل .tab
- **BottomNavigationBar** للتنقل.
- **AppBar** مع AcademicYearChip + WsConnectionDot + ThemeToggleButton.
- اشتراك في **WebSocket streams** (semester, year, sessionActivated...) لتحديث الـ state أو السحب من السيرفر.
- معالجة **session expired** عبر الـ AuthCubit state.

12.4 features/doctor/sections/doctor_overview.dart

لوحة home: الترحيب باسم الدكتور، إحصائيات (عدد المواد، عدد المحاضرات اليوم، عدد الطلبة)، محاضرات اليوم، pull-to-refresh.

features/doctor/sections/doctor_subjects.dart 12.5

قائمة المواد التي يدرّسها الدكتور مع:

- Search bar
- Level filter chips (1, 2, 3, 4)
- لكل subject: اسم، code، ال TA المعيّن (لو موجود) + زر assign/remove TA
- pull-to-refresh

features/doctor/sections/doctor_attendance.dart 12.6

(2373 سطر — أكبر ملف). يدير دورة حياة جلسة الحضور:

1. اختيار اللكتشر (اليوم الحالي → فلتري).
2. طلب فتح الكاميرا على ال host عبر `AttendanceRepository.requestCameraHost`.
3. انتظار الكاميرا (`waitForCameraStarted`) مع progress indicator.
4. **Active phase**: عرض الطلاب الجاي يأكّدوا واحد واحد عبر WebSocket SESSION_DATA updates.
5. **QR phase**: زر "Begin QR" → السيفر يبيث QR codes ليأكّد كل طالب من جواله.
6. **End session**: يحذف ال session + يحفظ التقرير + بيعت report saved event.
7. **AutomaticKeepAliveClientMixin** ليحافظ على ال state عند الانتقال لل tab الآخر.

ال enum `SessionPhase { none, active, confirming }` يحدد ال UI.

features/doctor/sections/doctor_reports.dart 12.7

قائمة التقارير السابقة للدكتور. لكل تقرير: subject, date, present count, attendance rate. أزرار: View Details, Download PDF (`PdfReportService.generateAndOpen`), Delete

ال Filters: subject, level, department, date range. يستمع لـ `reportSavedStream` لإضافة تقارير جديدة تلقائياً.

features/doctor/sections/doctor_profile.dart 12.8

بيانات الدكتور، صورة بروفایل (يقدر يغيرها عبر `ImagePicker` → multipart upload)، زر "Manage Teaching Assistants" (يفتح `DoctorTAManagement`).

features/doctor/sections/doctor_ta_management.dart 12.9

شاشة كاملة لإدارة المعيدين المشرف عليهم. عرض كل TA مع توجيه أساسي + الصلاحيات. للدكتور:

- تفعيل/تعطيل `ta.session.activate` لكل subject.
- تفعيل/تعطيل `ta.grades.manage` لكل subject.
- إعادة تعيين subjects للـ TAs.

يستمتع لـ taPermissionsStream عبر WebSocket لمزامنة لحظية.

features/student/student_screen.dart 12.10

الـ Shell الطلابي. 4 sections (Overview, Schedule, Grades, Profile). نفس بنية DoctorScreen لكن بدون أزرار التحكم. يستمتع لـ:

- sessionActivatedStream → تنبيه الطالب إن في session فعّالة لمادة عنده.
- gradeUpdateStream → تحديث الدرجات لحظياً.
- "registrationApprovedStream → snackbar "Subjects approved".
- "levelsPromotedStream → "Promoted to next level".

features/student/sections/student_overview.dart 12.11

Home الطالب: ترحيب، إحصائيات (GPA، السمستر، عدد المواد، عدد المحاضرات هذا الأسبوع)، محاضرات اليوم، قائمة محاضرات قريبة.

features/student/sections/student_schedule.dart 12.12

الجدول الأسبوعي. Tabs: All Days / Today / Day-by-Day. عرض الـ Lectures + الـ Sections (التمارين مع المعيد). فلترة بالـ search query وبالـ اليوم.

features/student/sections/student_grades.dart 12.13

عرض الدرجات per semester / per level. لكل grade: تفاصيل المكوّنات (midterm, oral, practical, distribution chart) يعرض total + letter + GPA points + (attendance, assignment, final). `distributionsCache_` (يجيب التقسيم من السيرفر مرة واحدة لكل (doctor, subject)).

في الأسفل: GPA الترم + GPA المرحلة + GPA التراكمي + Earned Credits.

features/student/sections/student_profile.dart 12.14

بيانات الطالب + صورة بروفایل (تعديل عبر multipart + ImagePicker). **QR Code** personal (يستخدم qr_flutter)، يتم رفع scan log عند مسحه في session attendance.

Change password form: current password (verify by attempting login)، new password، confirm

13. طبقة Widgets

الوظيفة	Widget
Chip بنفسجي صغير يعرض السنة الأكاديمية الحالية مع ValueNotifier listener.	AcademicYearChip
غلاف Skeletonizer مع shimmer effect مخصص للـ dark/light. SkeletonCardList تعرض 6 placeholder cards.	AppSkeleton + SkeletonCardList
helpers ثابتة: success / error / warning / info / custom. تستخدم floating Snackbar بـ rounded corners.	AppSnack
Container بـ radius 24 + InkWell + border + shadow حسب الـ theme.	CustomCard
Centered error UI مع Try Again button + message + icon.	CustomErrorWidget
الزر الرئيسي. linear gradient, height 56h, loading state. CircularProgressIndicator أبيض.	GradientButton
Card بسيط للمحاضرة (اسم، وقت، مكان، level، زر activate لو showAction=true).	LectureCard
wrapper بسيط حول SkeletonCardList.	LoadingIndicator
CachedNetworkImage داخل placeholder = أول حرف من الاسم. static evict(url) لمسح الـ cache بعد upload.	ProfileAvatar
كارد إحصائيات بـ gradient لون + رقم كبير + أيقونة + label.	StatCard
زر دائري بـ animated rotation + scale يقلب بين الوضع الفاتح والداكن.	ThemeToggleButton
توستات (success/error/info). أبسط من AppSnack (legacy).	ToastMessage
نقطة خضراء/حمراء بـ glow تعرض حالة الـ WebSocket. تستعلم كل ثانية عبر Stream.periodic.	WsConnectionDot

14. تدفق البيانات (Data Flow)

14.1 تسجيل الدخول

LoginScreen

- ↳ AuthCubit.login(username, password, isStudent)
 - ↳ AuthRepository.studentLogin / doctorLogin
 - ↳ ApiService.studentLogin / doctorLogin (HTTP POST)
 - ↳ Server returns { token, user }
 - ↳ SecureStorageService.saveUserData + saveToken
 - ↳ ApiService.setToken (in-memory)
 - ↳ AuthCubit emit AuthState.success
 - ↳ WebSocketService.connect()
- ↳ AuthGate isAuthenticated=true يري DoctorScreen / StudentScreen
- ↳ AuthGate listener → DataCubit.fullReload()

14.2 تفعيل جلسة حضور (Doctor)

DoctorAttendance.activateSession(lecture)

- ↳ POST /api/active-sessions (إنشاء session id)
- ↳ AttendanceRepository.requestCameraHost
 - ↳ WS message للسيرفر يبعث
 - ↳ face-api models يفتح صفحة الكاميرا + يحمل Host
- ↳ AttendanceRepository.waitForCameraStarted (polling 90s)
- ↳ phase = active
 - ↳ WS sessionActivatedStream يأتون من confirmed students يعرض الـ
- ↳ زر "Begin QR" → AttendanceRepository.beginQrPhase
- ↳ phase = confirming → QR الطلاب يأكدون بـ
- ↳ زر "End Session"
 - ↳ AttendanceRepository.deleteSession
 - ↳ AttendanceRepository.saveReport
 - ↳ WS reportSavedStream → DoctorReports يضيف للقائمة

14.3 تحديث الدرجة (Realtime)

Server (admin يعدل grade)

- ↳ WebSocket: { type: "GRADE_UPDATED", subjectId, data: {...} }
 - ↳ WebSocketService._handleMessage → gradeUpdateStream.add
 - ↳ StudentGrades يستمع → DataCubit.updateGrade(grade)

↳ `DataState.allGrades` يتغير
↳ `BlocBuilder` مباشرة `list` يعيد بناء الـ

15. ملاحظات تقنية مهمة

15.1 SSL / Self-signed certificates

`MyHttpOverrides` في `main.dart` يثق بشهادات SSL للـ `host` الحالي المضبوط في `ServerConfig`. ده ضروري لو السيرفر المحلي يستخدم `self-signed cert`. لو المستخدم غير الـ `URL`، الـ `trust` يتحدث تلقائياً.

15.2 Offline-first behavior

DataCubit عند فشل الـ `network`:

1. يحاول قراءة الـ `cache` (`CacheService`).
2. لو الـ `cache` فاضي أو فيه `session expired` → يعرض `error state`.
3. لو الـ `cache` موجود → يستخدمه (`snapshot` قديم لكن أفضل من شاشة فارغة).

15.3 Hot restart safety

GetIt registrations كلها بـ `AppProviders.value` guard. `isRegistered` يستخدم نفس الـ `instances` من GetIt بدل ما يعمل جديدة. النتيجة: `hot restart` بدون تكرار الـ `services` أو فقدان الـ `singletons`.

15.4 ScreenUtil + textScaler clamping

استخدام `.sp. / .h / w` يجعل التصميم `responsive`. الـ `clamping` على `[1.0, 1.2]` `textScaler` يمنع كسر الـ `layout` لو المستخدم رافع الـ `system font`.

15.5 Image cache eviction

بعد رفع `avatar` جديد، `ProfileAvatar.evict(url)` يمسح الـ `cache` في 3 مستويات (`cached_network_image`, `Provider`, `PaintingBinding`) لضمان ظهور الصورة الجديدة فوراً.

15.6 Robust JSON parsing

`Section.fromJson` يجرب أسماء مفاتيح متعددة (`...taName`, `ta_name`, `teaching_assistant_name`) لأن السيرفر بيرجع أسماء مختلفة في `endpoints` مختلفة. ده `anti-pattern` عموماً، لكنه ضروري هنا لأن السيرفر مش `under our control`.

16. ملخص ال Workflow للمذاكرة

1. المستخدم يفتح التطبيق → main.dart → ServerConfig.load → initServiceLocator → AuthCubit.init → (يستعيد الجلسة) DataCubit.init → (يحمل البيانات) WebSocketService.connect.
2. **AuthGate** يقرر الشاشة بناءً على authState.user.role.
3. **الشاشة الرئيسية** تستخدم BlocBuilder<DataCubit> لعرض البيانات وتسجل StreamSubscriptions على WebSocket streams.
4. **عند تفاعل المستخدم** (تسجيل، تعديل، حذف) الشاشة تنادي method على cubit، cubit ينادي repository، repository يستخدم ApiService أو http مباشرة، النتيجة <Result>T ترجع لل emit state → cubit جديد → BlocBuilder يعيد البناء.
5. **التحديثات اللحظية** (من admin/doctors آخرين) تأتي عبر WebSocket → ال stream المناسب → الشاشة المستمعة تستدعي cubit.refresh () أو cubit.updateX (...).
6. **ال Offline**: لو ال network فشل، CacheService يقدم آخر snapshot. لو نجح ال network، يتم تحديث ال cache.
7. **ال Token**: محفوظ في SecureStorage (مشفر hardware-backed). ApiService يحتفظ بنسخة in-memory للسرعة. عند logout، الاثنين يتمسحوا.

17. الخلاصة

المشروع يستخدم **API** ← **Service** ← **Repository** ← **Cubit** ← **UI** memorable architecture. كل طبقة لها مسؤولية واضحة وقابلة للاختبار منفصلة. ال packages المختارة مناسبة لحجم ومتطلبات المشروع بدون over-engineering. ال realtime عبر WebSocket مع reconnect logic متقن، وال offline-first cache يحسن تجربة المستخدم في ال scenarios الضعيفة. ال Models immutable مع copyWith، وال <Result>T يفرض معالجة الأخطاء على مستوى ال compiler.

— نهاية الوثيقة —